

P3820R1

Fix constexpr `uncaught_exceptions` and `current_exception`

Published Proposal, 2025-09-07

Author:

[Lénárd Szolnoki](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

1 Revision history

1.1 R1

2 Background

3 Proposal

4 Motivation

5 Design considerations

5.1 Definitions

5.2 Consistent runtime and compile time behavior

5.3 Keep `constexpr` where we can

5.4 Keep useful functionality

5.5 Naming

6 Implementation experience

7 Wording

7.1 [exception.syn]

7.2 [uncaught.exceptions]

7.3 [propagation]

References

Informative References

§ 1. Revision history

§ 1.1. R1

1. Changed title from "Split constexpr `uncaught_exceptions` into distinct runtime and consteval functions."
2. Added proposed changes to `current_exception`.
3. Added [design considerations](#).

§ 2. Background

[P3068] applied the `constexpr` specifier to the functions in the header `<exception>`.

`uncaught_exceptions` and `current_exception` are among these functions, however at runtime their return values depend on mutable global state. It is impossible for these functions to always return a constexpr value that is consistent with the runtime state. When evaluated in a constant expression, in current implementations of [P3068] `uncaught_exceptions` returns the number of uncaught exceptions in the current constant evaluation instead of the current thread. `current_exception` returns a null exception pointer if there is no currently handled exception in the current constant evaluation.

The inconsistency of the semantics of these functions depending on whether they are called from a constant expression or at runtime causes a breaking change when the functions are called in a constant evaluation.

`const` integer variables are usable in constant expressions when they are constant-initialized, so when initializing them the initializer is constant evaluated first, and if that fails then the variable is initialized at runtime. Consider:

```
#include <exception>

int i = 1;

int main () {
    try {
        struct S {
            ~S() {
                // x is initialized to 0 in trial constant evaluation
                const int x = std::uncaught_exceptions();
                i = x;
            }
        };
        S s;
        throw 0;
    } catch (...) {
        // b is initialized to false in trial constant evaluation
        const bool b = (std::current_exception() != nullptr);
        return i + b;
    }
}
```

The program returns 0, which is a breaking change compared to C++23, where it returns 2.

§ 3. Proposal

1. Remove the `constexpr` specifier from `uncaught_exceptions()`.
2. Add a `constexpr int consteval_uncaught_exceptions() noexcept` function with similar semantics to `uncaught_exceptions()`, but counting exceptions within the current constant evaluation.
3. Make `current_exception()` only constant when the currently handled exception was thrown in the same constant evaluation.
4. Add `constexpr int consteval_current_exception() noexcept` function with similar semantics to `current_exception()`, but only returning pointers to currently handled excep-

tions that were thrown in the current constant evaluation, otherwise returning null.

§ 4. Motivation

`uncaught_exceptions()` is useful to write scope guards and other patterns to conditionally execute code when normally exiting a scope or due to stack unwinding. Now that we have exception facilities available in constant evaluation, this functionality is similarly useful in that context. Putting the `constexpr` functionality in a separate `consteval` function makes this functionality available in constant evaluation without a breaking change.

`current_exception()` is also generally useful around exception handling, whether at runtime or in constant expressions. The breaking change involving `current_exception()` can be avoided by making it conditionally constant. The resulting function is still generally useful in many exception handling use cases. Putting the remaining `constexpr` functionality in a separate `consteval` function serves the remaining use cases.

§ 5. Design considerations

§ 5.1. Definitions

Semantically, evaluation of initializers of `constexpr` variables are sequenced normally on the same thread. I define the terms *currently handled exception* and *uncaught exceptions* so that they do not depend on whether the current point in execution is within a constant evaluation or at runtime. They both refer to the exception state in the current thread.

These definitions help to define the `constexpr` interfaces to be consistent across compile time and runtime.

```
int main() {
    try {
        struct S {
            ~S() {
                // there is one uncaught exception here
                constexpr int a = [] {
                    //there is still the one uncaught exception here
                    return 0;
                }();
            }
        };
    }
```

```

    S s;
    throw std::runtime_error{};
} catch (const std::runtime_error& e) {
    // The currently handled exception is a runtime_error object
    constexpr int b = [] {
        // The currently handled exception is still the same runtime_e
        return 0;
    }();
}
}

```

§ 5.2. Consistent runtime and compile time behavior

`constexpr` functions preferably have the property that they return consistent results at runtime and at compile time. This property makes it easier to reason about their behavior.

One standard library function that does not have this property is `is_constant_evaluated`. This function is explicitly designed to return different results within and outside of constant expressions. Despite this, it is still a source of confusion, even outside of trial constant evaluations.

The danger of confusing users is even higher for functions that are not designed to have different semantics at runtime and at compile time, and the difference is subtle.

§ 5.3. Keep `constexpr` where we can

We should allow constant evaluation where we can be consistent with the runtime behavior. This is impossible for `uncaught_exceptions()`, but it is possible for `current_exception()` in some cases. In particular when `current_exception()` is evaluated within a catch handler that is still in the same constant expression.

Example:

```

#include <exception>

// MAYBE_CONSTEXPR either expands to "constexpr" or nothing

int main() {
    MAYBE_CONSTEXPR int x = []{

```

```

    try {
        throw 42;
    } catch (...) {
        // the currently handled exception is an int object with value
        return *std::exception_ptr_cast<int>(std::current_exception());
    }
}();
return x; // returns 42;

```

§ 5.4. Keep useful functionality

It is worthwhile to introduce `consteval` functions with different names with subtly different semantics to the corresponding runtime functions. This fills the gap of `constexpr` functionality where we cannot keep the runtime and compile time behaviors consistent.

§ 5.5. Naming

The newly introduced functions are `consteval_uncaught_exceptions` and `consteval_current_exception`. The `consteval` in their name refer to the subtly different semantics to their runtime counterpart: the return values of these functions correspond to the current constant evaluation, not the current thread.

§ 6. Implementation experience

There is previous implementation experience for [\[P3068\]](#) including these two functions.

Splitting `uncaught_exceptions` into separate runtime and `consteval` functions is a simple change. Implementing `consteval_current_exception` is similarly simple.

A conditionally constant `current_exception()` can be implemented on top of the [\[P3068\]](#) implementation:

```

inline void nonconst() noexcept {}

constexpr exception_ptr current_exception() noexcept {
    exception_ptr ret = current_exception_P3068();
    if (ret == nullptr) {

```

```

    nonconst();
}
return ret;
}

```

§ 7. Wording

Wording is relative to [\[N5013\]](#).

§ 7.1. [\[exception.syn\]](#)

```

constexpr int uncaught_exceptions() noexcept;
constexpr int consteval_uncaught_exceptions() noexcept;

```

...

```

constexpr exception_ptr current_exception() noexcept;
constexpr exception_ptr consteval_current_exception() noexcept;

```

§ 7.2. [\[uncaught.exceptions\]](#)

```

constexpr int uncaught_exceptions() noexcept;

```

Returns: The number of uncaught exceptions (`[except.throw]`) in the current thread.

Remarks: When `uncaught_exceptions() > 0`, throwing an exception can result in a call of the function `std::terminate`.

```

constexpr int consteval_uncaught_exceptions() noexcept;

```

Returns: The number of uncaught exceptions (`[except.throw]`) in the current constant evaluation.

Remarks: When `consteval_uncaught_exceptions() > 0`, throwing an exception can result in a call of the function `std::terminate`.

§ 7.3. [\[propagation\]](#)

```

constexpr exception_ptr current_exception() noexcept;

```

Constant When: There is a currently handled exception that was originally thrown within the current constant evaluation.

...

constexpr exception_ptr constexpr current_exception() noexcept;

Returns: A null [exception_ptr](#) if there is no currently handled exception that was originally thrown within the current constant evaluation, otherwise an [exception_ptr](#) object that refers to the currently handled exception or a copy of the currently handled exception. The referenced object shall remain valid at least as long as there is an [exception_ptr](#) object that refers to it. If the function needs to allocate memory and the attempt fails, it returns an [exception_ptr](#) object that refers to an instance of [bad_alloc](#). It is unspecified whether the return values of two successive calls to [constexpr current_exception](#) refer to the same exception object.

If the attempt to copy the current exception object throws an exception, the function returns an [exception_ptr](#) object that refers to the thrown exception or, if this is not possible, to an instance of [bad_exception](#).

§ References

§ Informative References

[N5013]

Thomas Köppe. *Working Draft, Standard for Programming Language C++*. URL: <https://www.open-std.org/jtc1/sc22/wg21/prot/14882fdis/n5013.pdf>

[P3068]

Hana Dusíková. *Allowing exception throwing in constant-evaluation*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3068r6.html>